Houdini: Security Benchmarking of Container Confinement

***Abstract*—While container-based workloads are now a standard part of our cloud infrastructure, container security remains a challenging problem. Container confinement is a particularly pressing problem, as without it, a single vulnerable application can be used to compromise entire clusters of containers. While we have many technologies that can be used to secure containers, currently there is no easy way to determine whether a given Docker configuration provides even a basic level of protection. Here we present Houdini, a security test suite for container confinement. Much as network scanning tools can help catch misconfigured firewalls, Houdini can check whether a given Docker container configuration properly enforces confinement. While it can be used to test deployable containers, Houdini is optimised for testing and comparing container confinement technologies. Here we present the motivation, design, implementation, and initial results on running Houdini on a set of different container configurations. By providing a benchmark framework by which container confinement technologies can be evaluated, we believe Houdini can help foster the development of next-generation container confinement technologies.**

***Index Terms*—**

I. Introduction

Container-based workloads have become a cornerstone of modern cloud infrastructure, revolutionizing how applications are developed, deployed, and scaled, particularly within microservices architectures and DevOps workflows. Unlike hypervisor-based virtual machines (VMs), which each require a full operating system and rely on a hypervisor for hardware virtualization, containers are lightweight, share the host system's kernel, and incur minimal overhead. This efficiency makes them exceptionally well-suited for continuous integration and rapid scaling, allowing organizations to optimize resource utilization and reduce costs while ensuring quick, consistent deployments. Furthermore, containers enable seamless portability by packaging all userspace dependencies into a unified image, thereby avoiding the common problem of environment-specific discrepancies. Yet, despite containers' numerous benefits, they also introduce new security challenges, especially where container confinement is concerned.

Linux kernel features such as cgroups, namespaces, and mount points provide resource, process, and network isolation for containers. However, this separation is not foolproof. Kernel vulnerabilities, such as buffer overflows or privilege escalation flaws can allow a process to circumvent the very mechanisms designed to confine it. Misconfigurations can similarly open holes in container security, for instance by granting unnecessary capabilities, mounting sensitive host directories, or running containers in privileged mode. Finally, sophisticated attacks like container escapes exploit these weaknesses to break out of the container's resource and namespace isolation. Once an attacker gains elevated privileges or direct access to the host system, they can disrupt other containers, exfiltrate data, or compromise the entire infrastructure. Technologies like SELinux, AppArmor, and seccomp provide additional layers of security to avoid these issues, but are notoriously complex to configure and prone to misconfiguration. A best practices guide for container security [1] can help with configuring a container, but it offers no guarantee that the given configuration will perform as intended by the end-user. As container adoption skyrocketed across industries, particularly in multi-tenant cloud environments, these security challenges have become more pronounced.

To address the aforementioned challenges, the cloud industry has developed technologies like gVisor and Kata Containers [2], which isolate containers further by running them within lightweight VMs. While this approach is effective, gVisor-based containers bypass seccomp confinement by intercepting and emulating syscalls instead of directly blocking them through seccomp, which could result in security vulnerabilties if the emulation logic is flawed. By developing methods to audit and validate confinement mechanisms, container security could be simplified by reducing reliance on such solutions.

Test suites play a crucial role in verifying that systems meet specified requirements and function correctly under defined conditions. They are widely used across computing disciplines, from performance benchmarking to security assessment. For instance, the Standard Performance Evaluation Corporation (SPEC) [3] is best known for developing industry-standard benchmarks that evaluate CPU performance and power efficiency, while the Transaction Processing Performance Council (TPC) [4] establishes benchmarking standards for database performance. The Phoronix Test Suite [5] is a widely used benchmarking tool, particularly for Linux-based systems, providing automated performance evaluations across various hardware and software configurations.

Beyond performance testing, test suites are essential in software development to prevent regressions. Many organizations mandate that all code changes pass a comprehensive suite of tests before integration. In production environments, test suites help maintain reliability through uptime monitoring, ensuring that services remain operational. Additionally, specialized security-focused test suites play a vital role in detecting vulnerabilities in deployed systems. Tools such as network scanners aid in identifying misconfigurations, outdated software versions, and unintended service exposures, helping organizations proactively mitigate security risks.

Despite the widespread use of test suites in the aforementioned areas, a significant gap remains in the context of containers. There are currently no standardized or widely adopted test suites designed specifically to evaluate container confinement, functionality, or isolation effectiveness. This absence leaves organizations to rely on ad hoc methods, which are often inadequate for uncovering subtle misconfigurations or ensuring robust confinement. Given the growing reliance on containerized environments in modern infrastructure, the development of comprehensive test suites tailored for containers is imperative. Such test suites would not only address these critical gaps but also provide a reliable foundation for improving trust and security in containerized systems.

In this paper, we describe Houdini, the first test suite for verifying container confinement. Given a Docker container configuration (including the kernel, docker, and runc version), Houdini will instantiate the container in a standalone QEMU-based VM and perform multiple tests (or tricks, as we call them) to evaluate whether the container is susceptible to various forms of misconfigurations or failures. These tests explore vulnerabilities that could disrupt container functionality, such as, but not limited to improper resource allocation, inadequate isolation, or insecure filesystem setups. Houdini is written in Python and is easily extensible, providing an extension language for writing controlled test cases (which we refer to as "tricks") designed to evaluate whether a container confinement mechanism properly enforces security boundaries.

In this paper, we describe the motivation, design, and implementation of Houdini. We also present the results of case studies by showing how Houdini can be used to detect misconfigured containers. Our hope with Houdini is that it will help with the deployment of better confined containers and will support the development of new technologies to more reliably confine containers.

The rest of this paper proceeds as follows. In section II we define container confinment and describe the container confinement problem. In Section III, we explain our motivation, which includes the container confinement problem. We describe Houdini's design in Section IV. We describe the implementation of Houdini in Section V. In Section VI we present case studies showing how Houdini can detect basic misconfigurations, and explain the tricks Houdini currently implements and their associated vulnerabilities. Section VII describes related work. Section IX concludes our paper.

II. The Container Confinement Problem

Container confinement can be defined as the set of mechanisms and strategies used to isolate a containerized application from the host system and other containers. Effective confinement is critical for maintaining system security and stability, as it ensures that containers operate within defined boundaries by restricting their access to system resources, processes, networks, inter-process communication (IPC), filesystems, devices, and kernel interfaces.

A challenge with container confinement is that the level of confinement of a container is not fixed; it can vary based on the specific requirements or intentions of the user. In other words, the degree to which a container is isolated from the host system or other containers is customizable to meet the user's needs. At one end of the spectrum, one can lock a container down so tightly that it is almost entirely isolated from the host system, which minimizes potential attack surfaces but can also render the container incapable of performing key tasks. On the other hand, allowing a container too much freedom undermines the security benefits of containerization by exposing more privileges than necessary. The optimal configuration, however, is one that strikes a balance between maximum confinement (restricting access to system resources as much as possible) and function flexibility (enabling the container to perform its intended tasks). Achieving this balance is the core problem and goal of container confinement.

A standard methodology for assessing container confinement is crucial due to the complexity of modern systems and the intricate vulnerabilities they present. A formal approach involving rigorously evaluating and analyzing the security of the container system might seem ideal, however, it is impractical because container environments are highly complex, with interactions between kernels, runtimes, orchestration tools, and external dependencies that are difficult to model comprehensively. Moreover, many vulnerabilities stem from subtle details that a formal model might overlook. Instead, an empirical approach offers a practical alternative. Though it will never be exhaustive, the goal is not comprehensive coverage but rather establishing a baseline set of tests that demonstrate effective container confinement. This approach is achievable and can evolve over time as new vulnerabilities are identified, allowing the methodology to remain current. Starting with tests for isolation, privilege escalation prevention, and resource limitation enforcement provides a solid foundation. By focusing on demonstrating security effectiveness in critical areas, this methodology can enhance confidence in container confinement while enabling continuous improvement and adaptation to the evolving threat landscape.

III. Motivation

Houdini is designed to provide a systematic and reproducible approach to testing container confinement mechanisms. One of its primary goals is to evaluate whether container security features, such as namespaces, cgroups, seccomp, and Linux capabilities are effectively enforcing isolation. Many security tools rely on theoretical guarantees or static analysis, but Houdini focuses on empirical validation by executing controlled security tests that verify the correct enforcement of security mechanisms. By doing so, Houdini ensures that container security mechanisms are correctly configured.

Houdini is not only for testing container isolation but also as a means to optimize container configurations. By systematically simulating various tasks within different Docker containers, Houdini can help determine the minimal set of privileges required for a container to perform its intended function. For example, administrators can start with a container

configured with broad privileges and then incrementally reduce permissions, such as Linux capabilities, seccomp filters, or resource limits, while monitoring task success. Houdini's testing framework will identify the point at which functionality is maintained while unnecessary privileges are removed, thereby pinpointing the least privilege configuration that still allows the container to operate effectively. This approach is highly beneficial because it adheres to the principle of least privilege. Minimizing the privileges granted to a container reduces its attack surface, making it significantly harder for an attacker to exploit vulnerabilities.

Although runtime monitoring and anomaly detection, such as with eBPF-based tracing can provide immediate insights into container behavior, our goal isn't to implement an exhaustive, continuous surveillance system. Instead, we aim to focus on auditing and validating the underlying isolation mechanisms to ensure they are correctly configured and consistently enforced.

The following are additional goals of Houdini:

Identifying Gaps Between Intended Confinement Policies and Actual Enforcement: There is often a discrepancy between the security policies that administrators intend to enforce and what is actually upheld at runtime. These discrepancies generally fall into two categories: implicit policy problems and explicit policy problems.

An implicit policy problem happens when an administrator doesn't explicitly set a security rule, and the system's default behavior is used instead. These defaults might seem secure, but they can allow things the administrator did not expect, leading to gaps between what was intended and what actually happens. For example, Docker containers are, by default, connected to a shared internal bridge network. While this may prevent internet access, it still allows containers to communicate with each other. An administrator might see that outbound pings from a container are blocked and mistakenly believe the container is fully isolated from other networks. However, internal communication between containers may still be allowed unless it's explicitly disabled. This default behavior can create a false sense of security and introduce unexpected risks.

An explicit policy problem, on the other hand, occurs when a security policy is correctly specified but is later invalidated by other conflicting configurations. For example, an administrator might launch a container with `--network=none`, expecting it to be fully isolated from all network communication. However, if the container is also inadvertently given elevated privileges such as `--cap-add=NET_ADMIN` or `--privileged`, it can reconfigure its own network interfaces and regain connectivity. With the `NET_ADMIN` capability, processes inside the container gain access to low-level networking operations, such as creating virtual interfaces, assigning IP addresses, or modifying routing tables. A process could use commands like `ip link set eth0 up` and `dhclient eth0` to bring up a previously disabled interface and request an IP address from the host's network. In the case of a `--privileged` container, even more capabilities are granted, allowing near-complete access to the host's kernel features, including the creation of new network namespaces or interfaces, effectively bypassing the isolation intended by `--network=none`.

Challenging Misleading Security Assumptions with Empirical Testing: Container security is often discussed in terms of theoretical guarantees, with many security best practices based on assumptions rather than direct testing. For instance, documentation may state that a container running in unprivileged mode cannot escalate to root, but Houdini actively tests whether that is actually the case under different container configurations. By running real-world security validation instead of relying solely on security claims, Houdini helps challenge misleading assumptions and exposes where security mechanisms fail in practice. This is particularly important as container escapes and privilege escalation exploits continue to emerge, demonstrating that security controls do not always work as expected. By grounding security claims in measurable results rather than theory, Houdini helps researchers and practitioners develop more robust container security strategies.

In order to achieve the aforementioned goals, we designed Houdini with the following in mind:

1) *Reproducible Results.* A key goal of Houdini is to ensure that tests yield consistent and repeatable results. By controlling system variables and maintaining a structured testing environment, Houdini allows researchers and practitioners to compare results across different system configurations and security policies.
2) *Separation from the Host.* Houdini is designed to run tests inside a controlled containerized environment within a virtual machine (VM), ensuring that even if a trick triggers a security vulnerability, it does not compromise the host system running the tests. This design choice enhances the safety of security evaluations, preventing unintended side effects on the underlying infrastructure.
3) *Test Case Expressiveness.* Houdini tricks should be maximally expressive, such that some combination of steps can be used to achieve and test any desired result. It should be possible to define a new Houdini trick and modify existing tricks without modifying the Houdini binary. Moreover, it should be clear from reading a defined trick precisely what steps are involved, the consequences of each step passing or failing, and the overall nature of the exploit being tested.
4) *Focus on Observing Failures, Not Preventing Them.* Houdini is built to identify security weaknesses rather than defend against attacks.

IV. Design

Houdini is built using Python 3 and leverages a QEMU VM to provide a controlled testing environment for container security validation. Instead of running containers directly on the host system, Houdini starts a dedicated VM, ensuring that any security tests remain isolated and do not affect the host system. Within the Houdini configuration, one is able to specify the Docker version, kernel version, and runc version

to be installed into the VM. The VM is created with the help of Buildroot, which enables the construction of minimal, reproducible Linux environments tailored for security testing. This high level of configurability enables our goal of easily testing how different kernel and container runtime versions affect policy enforcement. It also enables regression testing across versions to detect when security behavior changes unexpectedly.

At the core of Houdini's architecture, a Flask-based server runs on the host operating system. This server acts as a bridge between the host and the VM, facilitating communication between them over a lightweight HTTP API. The Flask server is responsible for sending commands and configuration data to the VM and receiving trick execution results. When a trick is initiated, the Houdini client on the host instructs the VM to start a new trick. The VM then launches a container with the specified security configurations and executes the trick inside it.

When a trick is initiated, the Flask server exposes an endpoint that the Houdini client on the host uses to send a request containing the container configuration and the test logic to execute. The server then transmits this data into the guest VM. Inside the VM, a listener service receives this request, parses the configuration, and launches a container with the specified settings. The trick code is then executed within the container. Once execution is complete, the listener inside the VM collects the results, and sends them back to the Flask server. The Flask server then responds to the original request with the results, allowing the host to log the outcome.

If Houdini were compromised during testing, it would not necessarily pose a threat to the host system due to the design of the testing environment. Houdini is intentionally designed to operate within a controlled, containerized environment inside a VM. This means that even if a trick were to exploit a vulnerability within the containerized environment, the impact would be contained within the VM, isolated from the host system and other containers. The VM itself serves as an additional layer of security.

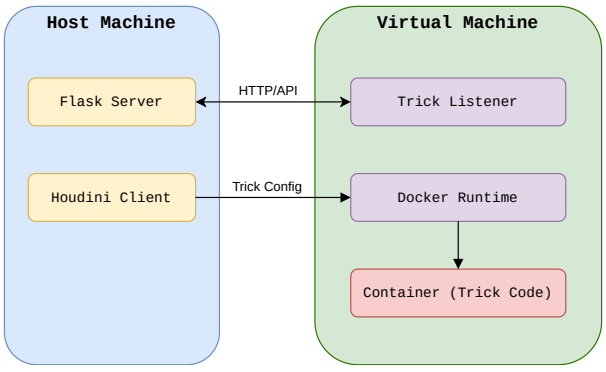


Fig. 1: Architecture of Houdini

V. Implementation

Tricks are divided into three separate files: a configuration file, a Python file, and a Dockerfile. Each `Houdini` trick begins with a configuration file (see Listing 1), which contains the Docker container's settings and environment configuration. The configuration file defines various Docker-specific parameters that are directly fed to the Docker API to start a container with the desired settings. Once the configuration is parsed, `Houdini` communicates these settings to the Docker API, using Docker's docker run command to initiate the container with these parameters. This ensures that the container is launched with the exact environment specified. This automation ensures the container is started exactly as specified, and that parameters such as network mode, mounts, security options, and user privileges are all controlled from a single file. This makes it easier to test, replicate environments.

Listing 1: Configuration file for a default Docker out-of-the-box security settings applied when a container is launched.

```
name: out-of-the-box docker configuration
dockerfile:
  - path: Dockerfile
dependencies:
  - server: False
docker_config:
  - network_mode: bridge
  - read_only: False
  - security_opt: []
  - pid_mode: null
  - cpu_shares: 1024
  - volumes : []
  - mem_limit: null
  - cpuset_cpus: null
  - cpu_quota: null
  - cpu_period: null
  - cap_add: []
  - cap_drop: []
  - privileged: False
  - user: root
  - pids_limit: null
  - ipc_mode: private
  - userns_mode: null
trick:
  - path: /houdini/tricks/OutOfTheBoxExample
```

The second component is the trick implementation, which can be written in any programming language and is executed inside the container. This file is responsible for carrying out the actual trick scenario. Currently, each trick implementation uses a print-based method to indicate whether the trick passed or failed. That is, after executing its logic, the script prints a human-readable message such as "Trick passed" or "Trick failed" to standard output. This message is then captured by the system and used to determine the result of the test. The definition of what constitutes a pass or fail is entirely controlled by the logic within the trick implementation itself. This allows the test writer to tailor the evaluation criteria based on the context of the trick.

Finally, the Dockerfile is used to configure and set up the container's environment. It specifies how the container should be built, including copying necessary files into the container and installing required dependencies. The Dockerfile

Listing 2: Exploit for CVE-2024-21616

```
import os

def check_shadow_access():
    path = "/etc/shadow"
    if os.path.exists(path):
        try:
            with open(path, 'r') as f:
                print("Able to read /etc/shadow:")
                print("Trick passed")
        except PermissionError:
            print("Permission denied to read
            ↪  /etc/shadow.")
            print("Trick failed")
        except Exception as e:
            print(f"Unexpected error: {e}")
            print("Trick failed")
    else:
        print("/etc/shadow does not exist.")
        print("Trick failed")

check_shadow_access()
```

also configures the container with the settings defined in the configuration file (see Listing 1), ensuring the container is set up correctly to run the trick.

This modular design, comprising of the configuration file, a trick implementation file, and Dockerfile provides flexibility to define and test various container escapes and vulnerabilities. It also allows users to easily customize or extend existing tricks by modifying any of the three components to suit their needs.

To illustrate how `Houdini` operates, a simple example trick is provided. This introductory trick attempts to read the /etc/shadow file inside the container. While not an actual exploit, this scenario serves as a baseline check for container isolation: under normal configurations, containers should not have permission to access this file. The purpose of this example is to demonstrate the structure of a trick using `Houdini` 's modular design, without relying on any specific vulnerability. Actual tricks and exploits are discussed in detail in the Evaluation section.

The Dockerfile defines the container's build environment, installing any necessary packages and copying the trick script into the container. In this example, the container is based on a minimal Python image, and the trick script is executed upon container startup (see Listing 4). The trick implementation itself is a Python script (see Listing 3) that attempts to open /etc/shadow. If the file is accessible and readable, it prints a partial contents and reports "Trick passed." Otherwise, it reports "Trick failed." This simple logic is sufficient to demonstrate Houdini's end-to-end flow from configuration to result interpretation.

VI. Evaluation

A. Case Study 1: Evaluating CVE-2024-21626

CVE-2024-21626 is a critical vulnerability in runc, the low-level container runtime used by Docker and other container engines. It allows a malicious container to escape its isolation and execute arbitrary commands on the host system.

The root cause of this vulnerability lies in how runc handles cgroups for containers, which are used to manage system resources like CPU and memory. During container creation, runc opens a file descriptor to the host's `/sys/fs/cgroup` directory to configure resource limits. This descriptor remains open throughout the container's lifecycle and is exposed inside the container via `/proc/self/fd/`, which lists all open file descriptors of the running process. The vulnerability arises when an attacker sets the working directory in the Dockerfile to point to one of these exposed descriptors, such as `/proc/self/fd/N`, where `N` refers to the specific file descriptor inherited from runc that points to the host's cgroup hierarchy. This causes the container's current working directory to resolve to a privileged host path. As a result, all subsequent relative file operations in the container are interpreted from the host's cgroup filesystem rather than the container's own root, allowing unintended access or modification of host resources.

This behavior is enabled by Linux's treatment of `/proc/self/fd/N` as dynamic symlinks to open file descriptors. When dereferenced, these symlinks resolve to the original host-side path, regardless of container filesystem restrictions. Because runc and Docker do not restrict access to inherited file descriptors or sanitize the working directory path during initialization, attackers can exploit this oversight to escape isolation. By simply changing the container's working directory to a leaked file descriptor, a malicious container can directly interact with host-managed control groups.

To test the exploitability of CVE-2024-21626, we designed a `Houdini` trick that executes the following steps on various Docker and runc versions as shown in Table I:

1) Set the working directory to `/proc/self/fd/<fd>`, pointing to a controlled file descriptor.
2) Attempt to break out of the container by leveraging the misconfigured working directory.
3) Validate whether commands can be executed on the host system.

The CVE for this vulnerability indicates that the affected runc versions are $\geq$ 1.0.0-rc93, $\leq$ 1.1.11. This suggests the vulnerability was introduced in runc version rc93 and fixed in version 1.1.12. However, empirical tests conducted by `Houdini` revealed a significant discrepancy with this assertion. Houdini's testing showed that earlier runc versions, such as `1.0.0-rc1`, `1.0.0-rc2`, `1.0.0-rc3`, and `1.0.0-rc4` also exhibited the same vulnerability. This discovery expands the scope of the CVE by revealing that the issue existed well before version `rc93`.

This discrepancy highlights a recurring issue in vulnerability disclosure processes: CVEs often rely on assumptions from static code analysis or limited testing. Without rigorous empirical validation across historical versions, critical edge cases can be missed. In the case of CVE-2024-21626, this oversight could have allowed vulnerable deployments relying on older runc versions to remain unpatched under the false belief that they were unaffected by this vulnerability.

TABLE I: runc Versions Against CVE-2024-21626

Docker Version	Runc Version	Leaked FD	Exploit Success
1.11.0	runc 0.0.9	-	✗
1.12.0	runc 1.0.0-rc1	/proc/self/fd/4	✓
17.03.0-ce	runc 1.0.0-rc2	/proc/self/fd/4	✓
17.06.1-ce	runc 1.0.0-rc3	/proc/self/fd/5	✓
17.09.0-ce	runc 1.0.0-rc4	/proc/self/fd/5	✓
-	runc 1.0.0-rc8	-	✗
-	runc 1.0.0-rc10	-	✗
-	runc 1.0.0-rc91	-	✗
20.10.17	runc 1.1.2	/proc/self/fd/9	✓
23.0.1	runc 1.1.4	/proc/self/fd/8	✓
23.0.5	runc 1.1.5	/proc/self/fd/8	✓
24.0.2	runc 1.1.7	/proc/self/fd/9	✓
24.0.5	runc 1.1.8	/proc/self/fd/9	✓
24.0.7	runc 1.1.9	/proc/self/fd/9	✓
24.0.0	runc 1.1.11	/proc/self/fd/9	✓
-	runc 1.1.12	-	✗
-	runc 1.1.13	-	✗
-	runc 1.1.14	-	✗

These findings highlight two key aspects of `Houdini` 's role in uncovering the true scope of the vulnerability. Firstly, it challenges misleading security assumptions, and secondly, it identifies gaps between intended and actual enforcement.

The security assumption outlined in the CVE is that the vulnerability exists only in runc versions $\geq$ 1.0.0-*rc*93, $\leq$ 1.1.11, suggesting that earlier versions are safe. This creates a false sense of security, leading users and system administrators to believe there is no urgency to patch or retire older versions. Rather than accepting the securty advisories claims, `Houdini` takes a more rigorous approach by empirically testing the behavior of the continaer to determine whether the vulnerability is present in other runc versions. In doing so, `Houdini` follows a fundamental principle of system assurance: vulnerabilities cannot be assumed absent, they must be revealed through observation. As CVEs often rely on static code inspection or version diffs to define the scope of a vulnerability, we argue that validation must also occur through dynamic runtime analysis. While the former methods are helpful, they have inherent limitations. For example, static analysis inspects code without execution, and version diffs highlight changes but cannot guarantee correct behavior. As a result, issues that only surface during real execution, especially under specific configurations may go unnoticed.

Listing 3: Trick Implementation for Case Study I

```
#!/bin/bash
relative_path='../../../../../'
target_file='houdini/tricks/HostMount/61240abcc5412ab'

if cd "$relative_path" 2>/dev/null; then
    echo "Successfully changed directory to: $(pwd)"

    if [ -f "$target_file" ]; then
        echo "File found: $target_file"
    else
        echo "File not found: $target_file"
    fi
else
    err_code=$?
    case $err_code in
        1)
            echo "Error: Permission denied"
            ;;
        2)
            echo "Error: No such file or directory"
            ;;
        *)
            echo "Unexpected error (exit code $err_code)"
            ;;
    esac
fi

echo "Docker version:"
docker --version || echo "Docker not found"

echo "runc version:"
runc --version || echo "runc not found"
```

Listing 4: Dockerfile for Case Study I

```
FROM ubuntu:16.04
WORKDIR /proc/self/fd/4
COPY host_mount.sh /host_mount.sh
RUN chmod +x /host_mount.sh
CMD ["/host_mount.sh"]
```

B. *Case Study 2 - Evaluating Host Process Manipulation*

In this evaluation, we examine whether a Docker container running as a non-privileged user with `UID=1000` can interact with and manipulate host processes that are also owned by `UID=1000`. Under a default Docker configuration, the PID namespace provides process isolation by restricting a container's visibility and access to processes outside its own namespace. However, when the container is launched with the `--pid=host` flag, it shares the host's PID namespace, allowing it to enumerate and interact with host processes. This configuration effectively bypasses one of the core isolation mechanisms provided by containers. To assess the security implications of this namespace sharing, we investigate whether process-level interactions, such as sending signals or attaching debuggers are permitted between the container and host processes when UID ownership is aligned.

TABLE II: Testing Environment

Component	Specification
Host Machine	20 Core CPU, 16GB RAM, Ubuntu 24.04
Virtual Machine	QEMU, 4 Core CPU, 12GB RAM, No Distribution
Docker Version	27.0.3
Kernel Version	6.7
Container Runtime	`runc` 1.1.13

To evaluate whether a non-root container can interact with and manipulate host processes, the container was started with a combination of various security configurations:

- `--pid=host`
- `CAP_KILL`
- `CAP_SYS_PTRACE`
- `Privileged Docker container`

The evaluation began by assessing whether a container launched with `--pid=host` could list host processes using `ps aux`. As expected, the container displayed all processes running on the host, confirming that sharing the host PID namespace fully exposes the host's process table to the container.

Following this, we tested whether a container process could send a `SIGCONT` signal to a host process owned by UID 1000.

The container was launched with `--pid=host`, allowing it to see and interact with host processes. Despite not being granted `CAP_KILL`, the signal was successfully delivered. This is because by default, both processes share the same UID and PID namespace, and under standard Unix permission checks, a process can send signals to another process it owns by UID without requiring special capabilities. However, if the target host process is owned by a different UID (another non-root user), `CAP_KILL` is required for the signal to be delivered. That capability must also be effective in the user namespace of the target process. Similarly, signaling host processes owned by UID 0 (root) remains restricted even with `CAP_KILL`, unless the container process itself is also running as root.

As shown in Table III, combining `--pid=host` with `CAP_SYS_PTRACE` enables nearly the same level of interaction with host processes as granting `--privileged` mode. Specifically, it allows a containerized process to list host processes, send signals, and attach a debugger to them, all key capabilities typically associated with privileged containers. However, unlike `--privileged`, this more targeted configuration limits access to only what's required, reducing the attack surface and following the principle of least privilege. These results demonstrate that it is unnecessary to over-privilege containers when finer-grained controls like capabilities and namespace sharing can achieve the same functional outcomes with significantly better security posture.

When the container is confined by AppArmor's `docker-default` profile, its ability to interact with host processes becomes significantly restricted. Specifically, `docker-default` explicitly denies critical system calls such as `signal` and `ptrace`, which are required for sending signals and attaching debuggers to other processes. As a result, even if the container shares the PID namespace with the host and possesses the necessary capabilities, these operations will fail.

However, including the `--privileged` flag when starting the container overrides AppArmor confinement by disabling the `docker-default` profile entirely. This lifts all AppArmor-imposed restrictions, allowing the container full access to the previously restricted system calls. As a result, operations that were previously blocked, such as sending signals or attaching debuggers to host processes are now permitted, even if AppArmor would normally deny them.

It is also important to note that granting capabilities such as `CAP_KILL` and `CAP_SYS_PTRACE` alone is not sufficient for interacting with host processes unless the container shares the host's PID namespace. Without the `--pid=host` flag, the container operates in its own isolated PID namespace, meaning it cannot see or reference host PIDs at all. As a result, any attempt to send signals or attach to host processes will fail.

This case study highlights a core strength of `Houdini`, which is its ability to systematically explore and identify the least privilege for a given operation. By allowing users to declaratively modify container configurations, `Houdini` makes it easy to test a wide range of privilege combinations in a repeatable and automated way. Instead of relying on trial-and-error or defaulting to the overly permissive `--privileged` mode, users can incrementally grant permissions and observe the effect of each change. This enables security practitioners, developers, and researchers to precisely determine the minimum set of permissions required for a given trick to succeed. As a result, `Houdini` not only serves as a testing framework for container escapes and misconfigurations, but also as a practical tool for privilege minimization and hardening container deployments.

This case study highlights the complexity of security policies in containerized environments, highlighting that the runtime behavior of security configurations is not always straightforward or predictable. While administrators may configure a container with `--pid=host` to enable the container to monitor host processes, there is an assumption that certain operations, such as sending signals or attaching debuggers remain restricted. However, these assumptions may not hold at runtime due to the interplay of various factors such as user namespaces and UID mappings.

When the container shares the same user namespace as the host (or has aligned UID mappings), and the container process runs under the same UID as the host processes, it can bypass security restrictions that would normally prevent such interactions. In this case, the container may be able to perform operations like sending signals or attaching debuggers to host processes—even without elevated capabilities like `CAP_KILL` or `CAP_SYS_PTRACE`. These capabilities, which are typically required for such actions, are not necessary when the container and host share UID mappings, as the container process is considered the same user as the host processes it targets.

The non-trivial nature of these runtime behaviors is why houdini performs empirical tests to validate configurations. By simulating real-world interactions between the container and host processes, it can identify whether these configurations inadvertently allow unauthorized actions. If the container can perform actions it should not be able to, despite policy expectations, this exposes a gap between the intended security policy and its actual enforcement at runtime.

C. Case Study 3 - Evaluating Docker Privileged Port Forwarding Restrictions

In this case study, we evaluate Docker's network isolation mechanisms to determine how well they enforce privileged port binding restrictions. Starting with Docker version 20.03, a significant change was introduced to Docker. They redefined unprivileged ports to start at 0 instead of 1024, effectively removing the need for root to bind to privileged ports. Table IV presents the results.

This case study highlights how discrepancies between Docker versions in handling privileged port binding can create gaps between administrators' intended confinement policies and actual enforcement. A change in Docker's handling of binding to privileged ports, such as the removal of the NET_BIND_SERVICE requirement starting from Docker

TABLE III: Ability for containers to interact with host processes

Security Configuration	--pid=host	CAP_KILL	CAP_SYS_PTRACE	--privileged	apparmor=docker-default	--pid=host + apparmor=docker-default
--pid=host	◇ △	◇ △	◇ △ ⋆	◇ △ ⋆	◇	◇
CAP_KILL	◇ △					◇
CAP_SYS_PTRACE	◇ △ ⋆					◇
--privileged	◇ △ ⋆					◇ △ ⋆
apparmor=docker-default	◇					◇
--pid=host + apparmor=docker-default	◇	◇	◇	◇ △ ⋆	◇	◇

The following tests were run as a non-root user with UID=1000 and were run against the same non-root process with UID=1000.
◇ List Host Process
△ Send Signal to Host Process
⋆ Attach Debugger to Host Process

TABLE IV: Ability for Container to bind to privileged ports

Docker Version	NET_BIND_SERVICE + root	NET_BIND_SERVICE + root-less	root	root-less	Default Container
19.03.9	✓	✗	✗	✗	✗
19.03.0	✓	✗	✗	✗	✗
20.10.0	✓	✗	✗	✗	✗
20.10.3	✓	✗	✗	✗	✗
23.0.0	✓	✓	✓	✗	✗
25.0.0	✓	✓	✓	✗	✗
27.0.1	✓	✓	✓	✗	✗

20.03, may allow containers to bypass restrictions even when policies are set to prevent it.

For example, administrators might configure a container to be restricted from binding to privileged ports, but if Docker is running a version where the NET_BIND_SERVICE capability is no longer required for root containers (like in Docker 20.03 and beyond), the container could still bind to these ports despite the policy specifying otherwise. This creates a mismatch between the intended policy (restricting privileged port binding) and the actual enforcement (where containers can still bind to those ports in certain Docker versions).

Houdini helps identify these gaps by systematically testing container configurations across different Docker versions. By simulating different Docker versions and security policies, Houdini verifies whether containers are correctly enforcing the intended restrictions on privileged port binding. For instance, Houdini could test whether containers configured to prevent binding to privileged ports are still able to bind to those ports in Docker versions 20.03 or later, where the security model has changed. If a container running with Docker 20.03 or newer can bind to privileged ports despite the configuration specifying otherwise, Houdini will detect this discrepancy and highlight the gap between the intended policy and actual enforcement.

This set of experiments evaluates how Docker's privilege model for binding to privileged ports (i.e., ports below 1024) has evolved across versions. Traditionally, binding to privileged ports required root access or the NET_BIND_SERVICE capability. However, newer versions of Docker introduced support for binding to these ports even in rootless containers, provided the necessary capability is explicitly granted.

By testing various combinations of user privileges (root vs. rootless) and capability settings across multiple Docker versions, this case study reveals which configurations are sufficient and which are excessive for achieving the goal of binding to a low-numbered port. It demonstrates that in modern Docker versions (23.0.0+), it is no longer necessary to run as root or grant all privileges to perform this operation. Instead, a least-privileged approach using a rootless container with NET_BIND_SERVICE is sufficient.

This case study shows that `Houdini` can empower Docker users to avoid unnecessary elevation of privileges, particularly avoiding full root access or the use of the –privileged flag, both of which expose containers to a broader attack surface. Houdini enables this kind of analysis by systematically testing Docker configurations and identifying the minimum set of privileges required for a given operation, helping developers and administrators adopt safer, more secure defaults.

VII. Related Work

At the time of this writing, to our knowledge, Houdini is the first comprehensive approach for evaluating and comparing *container confinement mechanisms*. Much of the container security literature to date has focused on either vulnerability scanning of container images, building offensive/attack tools for container escapes, or proposing best practices for securing container runtimes. While these papers, tools, and documents do not directly share our research objectives, they still broadly fit into the container security landscape. Additionally, research on VM escapes and sandbox escapes has provided valuable insights into isolation vulnerabilities, many of which relate to container security. Techniques used to break out of hypervisors and sandboxed environments often share similarities with container escape methods, which the importance of strong confinement mechanisms. The remainder of this section does not aim to exhaustively enumerate all container security tools and systems, but rather to highlight the various salient methods, including prior research on times the confinement of VMs and sandboxes have been escaped as it is related to work.

Vulnerability Scanners. Many free, open, and closed-source tools exist for identifying the presence of known vulnerabilities in container images. Shu *et al.* [6] developed a vulnerability scanning tool to scan Docker Hub container images at scale. They found that on average, official and community images

have concerning amounts of vulnerabilities (180+), and that these vulnerabilities remain unpatched for hundreds of days. Their study utilized the Docker Image Vulnerability Analysis (DIVA) framework, which automates the discovery, download, and analysis of over 300,000 Docker images. The findings revealed that more than 80% of both official and community images contained at least one high-severity vulnerability, and many images were not updated for extended periods. Moreover, vulnerabilities were often inherited from parent images to child images, further increasing security risks.

Best Practices. Users in search of (often high-level) advice on how to improve the security of their container deployments can consult one of many best practices guides available online. One such guide is the Center for Interent Security (CIS) Docker Benchmark [1]. In our experience, guides tend to offer generic advice (e.g., "*Only allow read access to the root filesystem*". This is problematic because it oversimplifies container security. Generic advice like "Only allow read access to the root filesystem" may seem straightforward, but it doesn't account for the specific context or unique configuration of an individual deployment. Such one-size-fits-all recommendations can lead users to believe that simply following these broad guidelines is enough to secure their systems, potentially leaving critical vulnerabilities unaddressed. In reality, effective container security requires a nuanced, tailored approach that considers the particular needs and threat models of each environment. Houdini fills the gap between generic advice and practical security by enabling one to objectively evaluate the effectiveness of your container confinement measures.

Offensive Tools. The offensive security community has developed a range of specialized tools aimed at identifying and exploiting vulnerabilities within containerized environments. Containers, while providing strong isolation, are not immune to attacks, and these tools focus on testing the robustness of container defenses. CDK[1] and DEEPCE[2] are two widely used, open-source container penetration testing toolkits that leverage a collection of known exploits to gain persistence, escape the container, and gather sensitive information about the container environment. These tools aim to bypass isolation mechanisms like namespaces, cgroups, and SELinux, often using a variety of aggressive techniques to identify weaknesses in container security. CDK specifically uses any available method to circumvent security measures, providing a broad toolkit to test the limits of a container's defense. Meanwhile, DEEPCE, developed by stealthcopter, focuses on privilege escalation and container enumeration, attempting to exploit specific weaknesses to achieve container escapes, thereby compromising the host system. In contrast, Houdini takes a more systematic and structured approach to container security testing. Rather than focusing solely on exploiting vulnerabilities, Houdini allows for the direct comparison of defensive techniques by clearly defining the testing environment, outlining the exploit steps, and documenting the expected outputs.

[1]https://github.com/cdk-team/CDK

[2]https://github.com/stealthcopter/deepce

Virtual Machine Escapes VM escapes occur when an attacker successfully breaks out of a virtualized confined environment, gaining unauthorized access to the host system. Given that hypervisors manage multiple VMs on shared hardware, such escapes can lead to host compromise, cross-VM attacks, and lateral movement within cloud environments. One of the most notable VM escape vulnerabilities is VENOM (CVE-2015-3456) [7], which exploited a flaw in the virtual floppy disk controller (FDC) implementation in QEMU, affecting platforms like Xen, KVM, and VirtualBox. Other significant VM escape vulnerabilities include Cloudburst (2009), which exploited VMware's virtual graphics driver; CVE-2018-6981 and CVE-2018-6982, which targeted VMware ESXi, Workstation, and Fusion [8]; and multiple VirtualBox vulnerabilities (2017) enabling guest-to-host code execution [9]. Additionally, side-channel attacks like MDS/ZombieLoad (2019) demonstrated how speculative execution flaws could be leveraged across VMs [10]. More recent research has uncovered weaknesses in encrypted virtualization, such as the SEVered attack (2018), which extracted plaintext memory from AMD SEV-protected VMs [11], and the Heckler attack (2024), which manipulated non-timer interrupts to compromise confidential VMs in AMD SEV-SNP and Intel TDX [12].

Sandbox Escapes Sandboxing is a critical security mechanism for restricting the execution of untrusted code, commonly used in browsers, mobile apps, and security tools. A sandbox escape occurs when an attacker bypasses these restrictions, executing arbitrary code with higher privileges and potentially leading to full system compromise. Modern web browsers heavily rely on sandboxing to isolate JavaScript execution from the underlying OS. However, several high-profile vulnerabilities have enabled sandbox escapes. For example, CVE-2019-5786 [13], a use-after-free vulnerability in the FileReader API of Google Chrome, allowed attackers to escape Chrome's sandbox and execute code on the host. Other notable sandbox escape vulnerabilities include CVE-2022-26706, which exploited a flaw in macOS's App Sandbox to execute code outside of restricted permissions [14]; CVE-2023-36719, a stack corruption vulnerability in Windows OS that enabled escape from the Chromium sandbox [15]; and CVE-2024-4040, which was actively exploited to break out of CrushFTP's Virtual File System (VFS) [16]. Additionally, vulnerabilities in Judge0, an online code execution system, exposed systems to full takeover by allowing attackers to escape its sandbox [17]. Research has also uncovered critical flaws in vm2, a JavaScript sandbox library, which could be exploited to bypass its security restrictions and gain remote code execution [18].

VIII. Discussion

Houdini's design not only addresses critical security challenges in container confinement but also lends itself well to continuous security testing in modern software development environments. Because it is both safe and modular, Houdini can be seamlessly integrated into automated systems, such as CI/CD pipelines, to provide ongoing validation of container

security. This integration is particularly valuable for the following reasons:

In dynamic environments where kernel versions, Docker releases, or container policies are frequently updated, security configurations can quickly become outdated or misconfigured. By integrating Houdini into the CI/CD pipeline, organizations can automatically trigger a full suite of confinement tests whenever any part of the container stack changes. This continuous validation ensures that every new deployment or update is accompanied by an up-to-date security assessment.

IX. Conclusion

In this paper, we presented Houdini, a security benchmarking framework designed to evaluate the effectiveness of container confinement mechanisms. As container-based workloads continue to dominate cloud infrastructures, ensuring that containers are properly isolated is paramount for maintaining security. Houdini offers a systematic and empirical approach to test and assess various container security configurations, highlighting potential misconfigurations that may compromise container isolation. Our evaluation demonstrated the ability of Houdini to detect critical misconfigurations, including those leading to privilege escalation. By providing a reliable and reproducible method for testing container confinement, Houdini helps bridge the gap between theoretical security assumptions and practical real-world behavior.

As container technologies evolve, it is essential to continuously assess and improve their security mechanisms. Houdini's modular design ensures that it remains adaptable to new security challenges and configurations, allowing for ongoing validation and refinement of container security practices. By offering an open framework for container confinement benchmarking, Houdini aims to support the development of more secure container technologies, ultimately contributing to a safer cloud infrastructure for the future.

References

[1] Cis docker benchmark. In *CIS Docker Benchmark*, 2024.

[2] Sari Sultan, Imtiaz Ahmad, and Tassos Dimitriou. Container security: Issues, challenges, and the road ahead. *IEEE access*, 7:52976–52996, 2019.

[3] John L Henning. Spec cpu2000: Measuring cpu performance in the new millennium. *Computer*, 33(7):28–35, 2000.

[4] Raghunath Nambiar, Nicholas Wakou, Forrest Carman, and Michael Majdalany. Transaction processing performance council (tpc): State of the council 2010. In Raghunath Nambiar and Meikel Poess, editors, *Performance Evaluation, Measurement and Characterization of Complex Systems*, pages 1–9, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.

[5] Martin Zboril and Vlasta Svatá. Performance comparison of cloud virtual machines. *Journal of Systems and Information Technology*, 2025.

[6] Rui Shu, Xiaohui Gu, and William Enck. A study of security vulnerabilities on docker hub. In *Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy*, pages 269–280, 2017.

[7] Cheick Abdoul-Kader and Shih-Hao Chang. A novel venom attack identification mechanism in cloud virtualization environment. *Communications of the CCISA*, 24(1):61–72, 2018.

[8] VMware Security Team. Vmware virtual machine escape vulnerabilities: Cve-2018-6981 and cve-2018-6982. 2018. Accessed on: March 3, 2025.

[9] Google Project Zero. 10 new vm escape vulnerabilities discovered in virtualbox. 2017. Accessed on: March 3, 2025.

[10] Stefan Lipp, Michael Schwarz, and Daniel Gruss et al. Mds: Microarchitectural data sampling. *IEEE Symposium on Security and Privacy*, 2019. Accessed on: March 3, 2025.

[11] Felix Dörre and Florian Piessens. Severed: Subverting amd's virtual machine encryption. *27th USENIX Security Symposium*, 2018. Accessed on: March 3, 2025.

[12] Amit M. and Thomas Eisenbarth. Heckler attack: Breaking amd sev-snp and intel tdx with non-timer interrupts. *arXiv preprint arXiv:2404.03387*, 2024. Accessed on: March 3, 2025.

[13] Philippe Laulheret. Analysis of a chrome zero day: Cve-2019-5786. 2019. Accessed on: March 3, 2025.

[14] Jonathan Bar Or. Uncovering a macos app sandbox escape vulnerability: A deep dive into cve-2022-26706, 2022. Accessed on: March 3, 2025.

[15] James Forshaw. Escaping the sandbox: A bug that speaks for itself, 2023. Accessed on: March 3, 2025.

[16] Satnam Narang. Cve-2024-4040: Crushftp virtual file system (vfs) sandbox escape vulnerability exploited, 2024. Accessed on: March 3, 2025.

[17] Ravie Lakshmanan. Sandbox escape vulnerabilities in judge0 expose systems to complete takeover, 2024. Accessed on: March 3, 2025.

[18] FortiGuard Labs. Vm2 sandbox escape vulnerability — outbreak alert, 2023. Accessed on: March 3, 2025.